

FYP Defence Preparation Guide: Desktop AI Assistant

Study guide for the proposal defence. Built from web research on papers, products and similar projects, mapped onto your project. Search date: 9 October 2026.

How to use this guide

1. Read **Part 1** (your pitch) and **Part 2** (the research landscape) first. They give you the vocabulary and the papers to name-drop.
2. Study **Part 3** (existing systems) and **Part 4** (problem statements and solutions). These are the core of the defence.
3. Part 5 explains the technical ideas behind your design. Parts 6 onward contain the mock questions and a last-minute cheat sheet.
4. **Verify before you cite.** I read abstracts and summaries, not every paper in full. Open each paper you plan to cite, confirm the claim, and note the page or section. Committees often ask "did you read it?"
5. Items marked [**decide**] are choices your team has not finalized. Have an answer ready for each, even if it is "we will decide by X date based on Y."

Part 1: Your project in one minute

30-second pitch. "Our project is a desktop AI assistant that watches only the application window the user chooses, detects what task they are doing, and checks the extracted information through a validation layer before sending it to an AI model. The assistant then points out likely mistakes, explains them and suggests fixes through chat, notifications and a desktop avatar. If it is not confident, it stays silent. We evaluate the system with precision, recall and F1 on a predefined set of test scenarios."

The three ideas that make your project defensible

1. **Validation before AI.** Most screen-aware assistants send raw screenshots straight to a large model. You insert an application-specific check in between, so the model sees only reliable, relevant information.
2. **Abstention (staying silent).** The assistant speaks only when it is confident. This trades some recall for higher precision, which is the right trade for an assistant people must trust.
3. **Measurement.** You do not just build it; you benchmark it with a test set and standard metrics, and compare against a baseline (raw screenshot to AI model, no validation).

The research question (say this exactly or close to it). "Does placing an application-specific validation layer between screen capture and the AI model improve the reliability of task suggestions, compared with sending raw screen content alone?"

Honest scope statement. A limited set of applications (for example VS Code and one or two Office apps) and a limited set of tasks. Not every application, and not autonomous control of the computer.

Part 2: Research landscape

This part groups the literature you found into themes. For each: what it is, what it found, how it connects to you, and what to say.

2.1 Surveys of GUI / computer-use agents

Paper	What it covers
<i>GUI Agents: A Survey</i> (Zhang et al., Findings of ACL 2025, pp. 22522-22538)	Categorizes benchmarks, evaluation metrics, architectures and training methods; proposes a framework of perception, reasoning, planning and acting; lists open challenges.
<i>A Survey on (M)LLM-Based GUI Agents</i> (arXiv 2504.13865, 2025)	Core architecture (perception, planning, action, memory); compares text-based perception (DOM/XML) with multimodal perception (screenshots plus tools like OCR and object detection).
<i>Large Language Model-Brained GUI Agents: A Survey</i> (arXiv 2411.18279)	Evolution from script-based automation to LLM agents; frameworks, data, Large Action Models, evaluation; notes that adding screenshots to text input improves performance.
<i>GUI Agents with Foundation Models: A Comprehensive Survey</i> (arXiv 2411.04890)	Taxonomy by input modality; discusses screen parsing and examples such as UI-TARS and AndroidWorld.

Key takeaways to remember

- Agents perceive the screen in four ways: accessibility APIs, HTML/DOM, screen-visual (screenshots), or hybrid. Your project uses screen-visual capture and may add application-specific structure (hybrid).

- Surveys report that current agents still have significant limitations in screen understanding and visual grounding. This supports your claim that raw screen interpretation is unreliable.
- Most of this literature is about agents that **act** (click, type). Your assistant **observes and advises**. That is a narrower, safer problem, and you can say so.

Say this: "Surveys of GUI agents show that perception from screenshots is a known weak point, so our design adds a validation layer rather than trusting raw interpretation."

2.2 Benchmarks: OSWorld

- **OSWorld** (Xie et al., NeurIPS 2024) is a benchmark of 369 tasks in real computer environments (Ubuntu primarily, with Windows and macOS support), covering web and desktop apps, file operations and multi-app workflows. It includes VS Code and office-style tasks, and each task has an execution-based checker.
- At publication, humans completed about **72.36%** of tasks while the best model reached only **12.24%**, mainly because of GUI grounding and operational knowledge problems.
- Progress since then has been fast. Vendor and leaderboard reports show agents around 60-77% by late 2025 and 2026, with some claiming to pass the human baseline (for example Simular's Agent S at 72.6%, AGI Inc.'s OS Agent at 76.26%, and a research system, Agent Alpha, at 77.29%). Treat these as self-reported; leaderboards note that setup differences matter, so be careful when comparing numbers.

Why it matters to you

1. It shows how benchmarking is done: tasks plus an automatic checker. You can copy that idea for your test scenarios (a scenario plus ground truth labels for the mistakes).
2. It shows the field moves quickly, so you should not claim that raw-screen AI is hopeless. Claim instead that **reliability, cost and trust** remain problems, and that validation gives measurable control.
3. OSWorld measures agents that *act*. It does not measure an *advisory* assistant's false alarm rate. That is part of your gap.

Say this: "OSWorld is the standard benchmark for computer-use agents. Our evaluation borrows its principle of scenario plus automatic ground truth, but measures advisory precision and recall instead of task completion."

2.3 Screen parsing and screen understanding

- **OmniParser** (Lu et al., Microsoft, arXiv 2408.00203): converts a UI screenshot into structured elements (bounding boxes and labels for interactable regions, with captions) using fine-tuned detection and captioning models. Secondary summaries

report GPT-4V accuracy on one sub-task rising from 0.705 to 0.938 when local semantics were added, and it improved results on ScreenSpot, Mind2Web and AITW. **Lesson: structuring the screen before the LLM sees it improves results.** This is the same philosophy as your validation layer.

- **ScreenLLM** (arXiv 2503.20978): introduces a compact textual "stateful screen schema" to represent GUI state and user actions over time, then trains models to understand user actions and intentions. **Lesson: compact structured representations beat raw pixels.**
- **Tool-augmented perception** (from the (M)LLM survey): combining a multimodal model with OCR and object detection tools. This is the category your extraction step falls into.

Say this: "OmniParser and ScreenLLM show that converting the screen into structured, compact information before the language model improves understanding. Our validation layer extends that idea from structure to correctness."

2.4 Proactive and screen-aware assistants (closest to your project)

Work	Venue / year	Relevance
Intent Assistant (INA) , "State Your Intention to Steer Your Attention"	arXiv 2510.14513, CHI '26	Monitors on-screen context against the user's stated intention, detects distraction and delivers gentle, non-intrusive nudges. Deployed with 22 participants. Shows proactive screen monitoring plus restraint is an accepted research direction.
Pointer Assistant , "Talk to the Hand" (Prasongpongchai, Pataranutaporn, Lertsutthiwong, Maes)	CHI '25	An LLM chatbot shown as a second mouse pointer that gives proactive feedback on on-screen tasks. A study with 220 participants (budget planning task) found users saw it as fun and helpful, but said its assertiveness could be improved. Supports your avatar / personality idea and the need for tone control.

Work	Venue / year	Relevance
ProAgentBench	arXiv 2602.04482, 2026	Defines a proactive agent as one that continuously monitors screen snapshots and initiates contact when it detects a need; provides a real-world benchmark. Directly supports your problem framing and your "when to speak" question.
AskEase	arXiv 2601.18092, 2026	An on-demand assistant giving context-aware step-by-step guidance for screen-reader users; in a within-subjects study with 12 users it improved task success and reduced workload. Shows guidance (not automation) helps users.
PExA (Bui et al., AAMAS demo, 2008)	2008	An early context-aware personal desktop assistant that tracked user workflows across instrumented desktop applications and made proactive suggestions. Shows the idea is old; what is new is LLMs.

Gap you can claim (carefully). These works focus on attention, usability, benchmarking proactivity or accessibility. The ones I found do not centre on **measuring whether an application-specific validation layer reduces wrong suggestions**. Phrase it as "in the work we reviewed" and finalize it after your own literature review; do not claim nobody has done it.

2.5 Hallucination and verification

- A survey of hallucination in LLM-based agents (arXiv 2509.18970) separates **self-verification** from **external validators**, and lists validator types: language-based, retrieval-based, execution-based and others. Execution-based validators check outputs by actually running code or plans.
- A comprehensive LLM hallucination survey (arXiv 2510.06265) groups mitigation into prompt, retrieval, reasoning and training-based approaches, and concludes that no single approach handles all failure modes, which supports domain-aware, task-adaptive pipelines combining complementary techniques.

- Practical guidance in the industry literature: ground the model on provided material, allow it to abstain, and add verification after generation.

Your mapping: the validation layer is an **external validator** placed *before* the model (input validation) and optionally after it (checking the model's claim against the extracted evidence). Abstention is the "allow it to say nothing" principle.

2.6 Hybrid rule-based plus LLM error detection

- Du et al., *Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry* (ICSE 2026 SEIP; arXiv 2601.18844): on a Tencent dataset of 433 alarms (328 false positives, 105 true positives), hybrid techniques combining static analysis and LLMs eliminated 94-98% of false positives while keeping high recall, at very low cost per alarm. They also report that false alarms cost developers around 10-20 minutes of manual inspection each.
- **ZeroFalse** (arXiv 2510.02534): treats static analyzer output as structured evidence that an LLM then adjudicates, keeping the analyzer's coverage while improving precision.

Why this is gold for your defence: it is evidence that **a deterministic checker plus an LLM beats either one alone** for mistake detection. For VS Code, your validation could be a linter or compiler diagnostics, with the LLM explaining and prioritizing. Note the direction: those papers use the LLM to filter the tool's alarms; you use the tool to constrain the LLM. Both are the same hybrid principle, and you should be ready to explain the difference.

Part 3: Existing products and projects

System	What it does	Where your project differs
Microsoft Copilot Vision (Desktop Share)	Lets the user share apps, windows or the desktop so the AI can answer questions and coach by voice or text.	Question-driven and vendor-specific; no validation layer you can measure; no personal avatar and memory model of your kind.
Google Gemini Live (screen sharing)	Real-time conversation where the model can see shared screen content.	Same pattern: raw visual input to a large model; reliability not separately controlled.

System	What it does	Where your project differs
Microsoft Recall	Periodically screenshots the screen and makes it searchable. After the 2024 privacy backlash it was delayed and made opt-in. UK ICO raised concerns; later testing reported that sensitive-data filtering was not reliable.	A warning, not a competitor. Shows why user-selected window, start/stop control and local handling matter. See privacy answers in the Q&A.
Open-LLM-VTuber (GitHub)	Open-source AI companion with a desktop-pet mode, Live2D avatars, voice, screen and camera perception.	Strong on avatar and companionship; not focused on validated mistake detection or evaluation metrics. Good evidence that the avatar part is technically feasible.
Clicky for Windows (GitHub)	A small AI buddy beside the cursor that explains what is on screen and points at things, with multiple model backends, including local models.	Tutoring from screenshots; claims to describe only what it sees, but no measured validation layer.
GitHub Copilot / IDE assistants	In-editor code suggestions and chat.	Tied to one application, uses direct editor access, not a cross-application, user-controlled monitoring assistant.
Classic Clippy / Office Assistant	Rule-based proactive help with an animated character.	Famous for being annoying; it interrupted too often. This is exactly why you add confidence-based silence and tone control.

Lessons to cite in the defence

1. **Privacy sinks products (Recall).** So: select a window, show monitoring status, allow stop, avoid storing screenshots, redact sensitive content.
2. **Annoying proactivity sinks products (Clippy; Pointer Assistant participants wanting a better balance of assertiveness).** So: abstention, tone settings, a playful mode that can be switched off.

3. **Avatars and desktop pets are feasible** (Open-LLM-VTuber). So: the avatar is a presentation layer, not a research risk.
4. **Big vendors already offer screen-aware chat.** So do not claim novelty for "AI that sees your screen." Claim novelty for **validated, measured, window-restricted guidance**.

Part 4: Problem statements and how the project solves them

#	Problem	Evidence	How your project answers it
P1	Unreliable AI suggestions from raw screens. Models misread screens or hallucinate, producing confident but wrong advice.	GUI-agent surveys cite grounding limits; hallucination surveys; OSWorld's initial 12% vs 72% gap.	A validation layer filters and verifies extracted information before the AI sees it, and the pipeline is measured against a raw-screen baseline.
P2	Interruption and false alarms erode trust. An assistant that flags non-mistakes gets ignored or disabled.	Static-analysis false-positive literature (10-20 minutes per false alarm); Clippy history.	Confidence-based abstention plus precision-focused evaluation; false positive rate is a reported metric.
P3	Context switching. Users copy content into a chatbot and explain the situation manually.	Existing chat assistants need manual input.	The assistant detects the application and task and receives the relevant content automatically from the chosen window.
P4	Privacy risk of always-on screen monitoring.	Recall backlash.	User-selected window only , start/stop control, only validated information leaves the machine, per-user data separation, API keys kept on the backend.
P5	No measurable reliability for assistants. Vendors rarely report precision/recall for advisory suggestions.	Benchmarks focus on task completion (OSWorld), not advisory false alarms.	A test set of scenarios with ground truth; reported precision, recall, F1, FPR, FNR and response time.

#	Problem	Evidence	How your project answers it
P6	Generic, impersonal assistants. They forget preferences and tasks.	Personalized-assistant literature, user needs studies such as INA's formative study.	Per-user memory and recurring reminders , with isolated accounts.
P7	Chat boxes feel detached from work. Notifications in separate windows are easy to miss, and tone is one-size-fits-all.	Pointer Assistant study (participants liked the visual companion and asked for tone balance).	An interactive desktop avatar with states, plus adjustable tone and optional playful mode.
P8	Support limited to one vendor's apps.	Copilot works in Microsoft apps; IDE assistants work in IDEs.	A cross-application design where each supported app plugs in its own detector and validator, starting with a small set.

How to answer "what is the single most important problem you solve?" P1 and P2 together: "making screen-based AI suggestions trustworthy enough to be useful, and proving it with numbers."

Part 5: Technical primer (know your design)

5.1 Pipeline and why each stage exists

Stage	Purpose	Typical options [decide]
Window selection and capture	Restrict monitoring to one user-chosen window; sample periodically, not continuously	Electron's screen/window capture APIs (check the current Electron docs for the recommended call, as the API has changed across versions); capture every N seconds or on window-change events
Application and task detection	Decide which supported app is open and what the user is doing	Window title and process name (cheap, reliable for app detection); screenshot classifier or rules for the task

Stage	Purpose	Typical options [decide]
Information extraction	Turn pixels into usable content	OCR (for example Tesseract.js), a vision-capable model, or a structured source where available
Validation	Confirm the extracted information is relevant and reliable	Rules, linters/compiler, format checkers, consistency checks, confidence thresholds
AI analysis	Explain the issue and suggest a fix	External AI API with a prompt restricted to the validated evidence
Output and silence	Decide whether to speak, and how	Confidence threshold; avatar, notification or chat

Be ready to defend the order. Validation sits before the model so that the model only sees evidence that passed a check. It reduces cost (fewer API calls), reduces hallucination (less noise) and makes errors traceable (you can tell whether a bad suggestion came from extraction, validation or the model).

5.2 Validation: concrete ideas per application

Your proposal says the method is to be researched. Having candidate answers prepared will make you sound far more credible. These are design suggestions for you to evaluate, not claims from the papers.

Application	Example tasks	Possible validation
VS Code	Syntax errors, undefined variables, missing imports, simple logic mistakes in a chosen language	Run the extracted code through a parser, linter or compiler (for example a Python syntax check, or ESLint for JavaScript). Only issues confirmed by the tool, or by tool plus model agreement, trigger a suggestion. This is an <i>execution-based / rule-based</i> validator.
Excel	Formula errors, inconsistent formulas, wrong ranges, mixed data types	Pattern rules on extracted cell content (for example error values such as #DIV/0!, formulas that differ from neighbours, text stored as numbers).

Application	Example tasks	Possible validation
Word	Spelling and grammar, inconsistent heading styles, formatting problems	Spell/grammar checkers and style-consistency rules.
PowerPoint	Too much text per slide, inconsistent fonts, missing titles	Simple layout rules and thresholds.

Advice: pick tasks where ground truth is easy to define and verify automatically. Syntax errors in code and formula errors in Excel are far easier to label than "is this slide persuasive?". Three or four well-chosen tasks, evaluated properly, beat ten vaguely evaluated tasks.

5.3 The screen-versus-API question (expect this one)

Many apps expose structured data: VS Code has an extension API and diagnostics, Office has automation interfaces, Windows has accessibility (UI Automation) trees. A sharp examiner will ask why you capture the screen at all.

Good answer. "Screen capture works for any application without installing plugins or needing vendor APIs, which is the generality we want to study. Structured sources are more accurate but application-specific. Our design is hybrid: we start from the screen, and where a cheap structured source exists we use it as part of validation. We will compare the two as part of the evaluation if time allows." The literature supports this: GUI-agent surveys describe accessibility-based, DOM-based, screen-visual and hybrid perception, and note that agents converge on combining visual and structural information.

5.4 Confidence and abstention

The assistant needs a rule for when to stay silent. Possible signals **[decide]**:

- OCR confidence is below a threshold.
- The validator and the model disagree.
- The app or task was not detected with confidence.
- The same issue was already reported and the user ignored it.

Tune the threshold on a validation subset, then report the trade-off: a higher threshold raises precision and lowers recall. A small chart of precision and recall versus threshold makes an excellent defence slide.

5.5 Metrics (formulas and a worked example)

Define for each test scenario whether a real mistake exists (ground truth) and whether the system reported it.

- **True positive (TP):** a real mistake, and the system reported it correctly.
- **False positive (FP):** no real mistake (or a different one), but the system reported one.
- **False negative (FN):** a real mistake the system missed (including when it stayed silent).
- **True negative (TN):** no mistake, and the system correctly stayed silent.

Metric	Formula	Plain meaning
Precision	$TP / (TP + FP)$	Of the alerts it raised, how many were right?
Recall	$TP / (TP + FN)$	Of the real mistakes, how many did it catch?
F1-score	$2 \times P \times R / (P + R)$	Balance of precision and recall.
Accuracy	$(TP + TN) / \text{all cases}$	Overall correctness; misleading if most cases are "no mistake".
False positive rate	$FP / (FP + TN)$	How often it raises a false alarm on clean cases.
False negative rate	$FN / (FN + TP)$	How often it misses a real mistake.

Worked example. Suppose 100 scenarios: 60 contain a mistake, 40 do not. The system reports 50 correct mistakes (TP = 50), misses 10 (FN = 10), raises 5 false alarms (FP = 5) and stays silent correctly on 35 (TN = 35).

- Precision = $50 / 55 = 0.91$
- Recall = $50 / 60 = 0.83$
- F1 = $2 \times 0.91 \times 0.83 / (0.91 + 0.83) = 0.87$
- FPR = $5 / 40 = 0.125$; FNR = $10 / 60 = 0.17$
- Accuracy = $85 / 100 = 0.85$

Be ready to explain **why you care more about precision** for an assistant: false alarms cost the user's trust and attention (the static-analysis literature reports 10-20 minutes lost per false alarm), while a missed mistake is the status quo without the tool.

The standard reference for these measures is Sokolova and Lapalme (2009), *A systematic analysis of performance measures for classification tasks*.

5.6 Privacy and security design

- Monitor only the selected window; show a clear indicator while monitoring; one-click stop.
- Do not store raw screenshots by default; process in memory and discard.
- Send only validated, minimal text to the AI API; consider masking obvious secrets (keys, passwords, card numbers) before sending.
- Keep the API key on the backend, never in the Electron renderer; use environment variables or an OS credential store [**decide**].
- Hash passwords (for example bcrypt), use per-user data separation in SQLite, and use parameterized queries to prevent SQL injection.
- Be honest that sending content to an external API has privacy implications, and offer a local-model option as future work.

Part 6: Evaluation plan (so you can answer "how will you prove it works?")

1. **Define supported tasks.** For each, define what counts as a mistake.
2. **Build the test set.** For each app and task, create scenarios: some with a seeded mistake, some clean. Aim for a balanced set, for example 30 to 50 scenarios per task (state the target, adjust to feasibility). Label ground truth by hand.
3. **Run three configurations.**
 - **Baseline A:** raw screenshot sent straight to the AI model.
 - **Baseline B (optional):** extracted text sent without validation.
 - **Proposed:** extraction plus validation plus AI, with abstention.
4. **Measure** precision, recall, F1, FPR, FNR, and response time (end-to-end latency per suggestion).
5. **Analyze errors.** Categorize failures by stage (capture/OCR, detection, validation, model) so you can say where the system breaks.
6. **Compare** the proposed configuration with the baseline(s). If validation improves precision at an acceptable recall cost, the hypothesis is supported. If not, report that honestly: a negative result with good analysis is still a valid FYP finding.

Threats to validity to mention: small test set; scenarios authored by the team (bias); one AI model; OCR quality varying with font, theme and resolution.

Part 7: Risks and honest limitations

Risk	Mitigation
Scope creep (avatar, memory, reminders, accounts, research, evaluation all at once)	Treat monitoring, validation and evaluation as the core; keep avatar, reminders and playful mode simple and time-boxed.
OCR errors on code or spreadsheets	Prefer structured sources where available; crop to regions of interest; confidence thresholds.
API cost and latency	Validate first so fewer calls are made; sample periodically; cache.
AI provider changes or outage	Keep the provider behind an interface; have a fallback or a local model option.
Small test set weakens conclusions	Report confidence intervals or at least counts; be explicit about limits.
Privacy objections	Window restriction, opt-in, stop control, no screenshot storage.
Application updates break detection	Limit to a few apps; version-pin during evaluation.

Part 8: Mock defence questions and model answers

These are model answers, not scripts. Put them in your own words, and adjust the **[decide]** items to whatever your team actually chooses. Practise saying each one aloud in under 45 seconds.

A. Motivation and novelty

Q1. ChatGPT, Copilot Vision and Gemini can already see my screen. What is new in your project?

A: Those tools are general question-answering systems that pass screen content to a large model. We do not claim to invent screen-aware AI. Our contribution is a **validation layer between the screen and the model, restricted to a user-chosen window, with confidence-based silence and a measured evaluation**. The question we study is whether that validation improves reliability compared with sending raw screen content. Vendors rarely publish precision and recall for advisory suggestions; we will.

Q2. Why is this a Software Engineering FYP and not just an API wrapper?

A: The engineering work is the pipeline around the model: window capture, app and task detection, extraction, validation, confidence handling, a multi-user backend with isolated data, scheduled reminders, a desktop avatar, and an evaluation harness. The AI model is one component. The research part is designing and evaluating the validation layer, which a plain wrapper does not do.

Q3. What problem are you actually solving?

A: Two things. First, screen-based AI suggestions are often unreliable, because models misread screens or produce confident wrong answers. Second, assistants that raise false alarms lose user trust. We reduce both by validating evidence before the model sees it, letting the assistant stay silent when unsure, and measuring the result.

Q4. Who is the target user?

A: Primarily students and junior developers or office workers who use VS Code and Office applications and benefit from immediate feedback on mistakes and from reminders. Secondary users are anyone who wants a personal assistant with memory.

Q5. What is your research gap?

A: In the work we reviewed, screen-aware assistants are evaluated mostly on task completion (for example OSWorld), usability, or attention management (for example the Intent Assistant study). We found little that centres on measuring how an application-specific validation step changes the precision and recall of advisory suggestions. We will finalize the gap statement after the full literature review.

B. Architecture and technical decisions**Q6. Why validate before the AI instead of just using a better model?**

A: Better models help, but they do not remove hallucination, and surveys conclude no single mitigation covers all failure modes. Validation is cheap, deterministic where possible, explainable and testable. It also cuts API calls and cost. Hybrid approaches support this: an industrial study from Tencent found that combining static analysis with LLMs removed 94 to 98 percent of false positives while keeping high recall.

Q7. What exactly is your validation mechanism?

A: It depends on the application and is being finalized [**decide**]. For VS Code we plan to run extracted code through a parser or linter and report only issues the tool confirms, or where tool and model agree. For Excel, rule checks on formulas and values. The common principle is an application-specific check that either confirms or rejects the information before it reaches the AI.

Q8. How do you get text from a screenshot reliably?

A: Options are OCR, a vision-capable model, or structured sources such as editor or accessibility data where available **[decide]**. We know OCR can fail on small fonts, dark themes or code symbols, so we crop to the relevant region, use confidence scores, and abstain when extraction confidence is low. OCR errors are one of the failure categories we will measure.

Q9. Why not use the VS Code or Office APIs, which are more accurate?

A: They are more accurate but application-specific and require integration for each vendor. Screen capture is general. Our design is hybrid: we begin from the screen and use structured sources as part of validation where cheap. The surveys on GUI agents describe the same trade-off between accessibility, DOM, visual and hybrid perception.

Q10. How do you detect which application and task the user is performing?

A: Application detection uses window metadata such as title and process name, which is cheap and reliable, combined with content cues. Task detection uses rules or a lightweight classifier on the extracted content **[decide]**. We limit the number of supported tasks so detection can be measured.

Q11. How often do you capture the screen, and what about performance?

A: Periodic sampling or change-triggered capture, for example every few seconds, not continuous recording **[decide]**. We skip analysis when the content has not changed, and validation reduces the number of API calls. Response time is one of our measured metrics.

Q12. Why Electron, React and Node.js?

A: One JavaScript stack across desktop shell, interface and backend, a large ecosystem, access to screen capture and native notifications, and transparent always-on-top windows for the avatar. The trade-off is memory use compared with a native app, which we accept for faster development.

Q13. Why SQLite?

A: It is embedded and needs no server, which suits a single-machine desktop application with per-user data. If the system needed many concurrent users across devices, we would move to a client-server database.

Q14. Which AI model or provider will you use?

A: Not finalized **[decide]**. We will compare providers on cost, latency, vision support and API terms, and keep the provider behind an interface so it can be swapped. Final choice will be documented with the criteria.

Q15. What happens if the AI API is down or too slow?

A: The system degrades gracefully: validation results (for example linter findings) can still be shown without AI explanation, with a timeout and a retry. We also treat latency as an evaluated metric.

C. Evaluation and research methodology

Q16. How will you evaluate the system?

A: We build a test set of predefined scenarios per supported task, some with seeded mistakes and some clean, with hand-labelled ground truth. We run the proposed pipeline and a baseline that sends raw screenshots directly to the model, and compare precision, recall, F1, false positive and false negative rates, and response time.

Q17. What is the difference between precision and recall, and which matters more here?

A: Precision is the fraction of reported mistakes that are real; recall is the fraction of real mistakes that were reported. For an assistant, precision matters more, because false alarms make users ignore or disable it, while a missed mistake leaves the user where they started. That is why we include abstention. We still report both and the F1-score.

Q18. Why is accuracy not enough?

A: If most scenarios contain no mistake, a system that never reports anything can score high accuracy while being useless. Precision, recall and false positive rate expose that.

Q19. How big is your test set, and is it enough?

A: Our target is a balanced set of seeded-mistake and clean scenarios per task, for example tens per task [**decide**]. It will not support strong statistical claims, so we will report counts and limitations clearly. The goal is a controlled comparison between configurations, not a general benchmark.

Q20. How do you avoid bias in a test set you made yourselves?

A: We cannot remove it entirely. We reduce it by writing scenarios from common real mistakes, having someone outside the team review the labels, mixing difficulty levels, and keeping part of the set unseen during tuning. We will list this as a threat to validity.

Q21. What if your validation layer does not improve results?

A: That is still a valid finding. We would analyze where it fails by stage (capture, extraction, detection, validation, model) and report whether the cost comes from lower recall or from poor validators. A well-analyzed negative result is acceptable research.

Q22. How do you compare with existing systems?

A: Not by claiming to beat commercial tools on their benchmarks. We compare against a controlled baseline on the same scenarios with the same model: raw screen input versus validated input. That isolates the effect of our contribution. OSWorld is the reference for how agent benchmarks are built, and we adopt its idea of scenario plus automatic checking.

Q23. What is OSWorld, and why is it relevant?

A: OSWorld is a benchmark of 369 tasks in real computer environments with execution-based checkers. At publication, humans completed about 72 percent of tasks and the best model about 12 percent; later agents reportedly reached or exceeded the human baseline. It measures agents that perform tasks, whereas our assistant advises, so we borrow its methodology rather than its tasks.

D. AI, reliability and ethics

Q24. How do you prevent hallucinations?

A: We cannot eliminate them. We reduce them by grounding the model on validated evidence only, instructing it to answer from that evidence, allowing abstention, and optionally checking the model's claim against the evidence afterwards. Then we measure the remaining error rate.

Q25. How does the assistant decide when to stay silent?

A: With a confidence rule combining extraction confidence, detection confidence and agreement between the validator and the model [**decide**]. We tune the threshold on a validation subset and report the precision and recall trade-off.

Q26. Is monitoring someone's screen a privacy problem?

A: It can be, which is why we design against it. Microsoft's Recall was delayed and made opt-in after criticism. We monitor only a window the user selects, show when monitoring is active, allow stopping at any time, do not store raw screenshots by default, send only validated minimal content to the API, and separate data per user. We also acknowledge that sending content to an external API remains a risk and propose local models as future work.

Q27. What if sensitive information such as passwords appears in the selected window?

A: We apply masking of obvious secrets before anything is sent, let the user pause monitoring, and recommend not selecting windows that display sensitive data. We will state clearly that automatic detection of sensitive data is not perfect; reports on Recall found its filtering unreliable in some tests.

Q28. How do you secure the API key?

A: The key lives on the backend, never in the renderer or in source control; it is loaded from environment variables or an OS credential store **[decide]**. The desktop client calls our backend, which calls the AI provider.

Q29. Could the assistant give harmful advice?

A: It only advises; it does not control the mouse, keyboard or files, so the user stays in control. Messages are framed as suggestions with an explanation, and the assistant stays silent when uncertain.

Q30. Is the playful mode appropriate?

A: It is optional, off by default, only triggers on clear mistakes, never when uncertain, and the user controls tone. Studies of proactive visual assistants (for example the Pointer Assistant study) found users liked a fun companion but wanted better control over assertiveness, which motivates tone settings.

E. Features, scope and feasibility

Q31. Which applications and tasks will you support?

A: VS Code and one or two Office applications **[decide]**, with a few tasks each, for example syntax and undefined-variable errors in code and formula errors in spreadsheets. We will finalize the list by a stated date, based on feasibility of validation and ground-truth labelling.

Q32. This seems like a lot for two people. What is the core?

A: The core is monitoring, detection, validation, AI analysis and evaluation. Accounts, memory, reminders and the avatar are supporting features built with simple implementations. The playful mode and extra avatars are optional and will be dropped first if time is short.

Q33. How will the avatar work technically?

A: A transparent, always-on-top Electron window displaying animated sprites or a rendered character, with states such as sitting, standing and walking, and a speech bubble for messages. Existing open-source projects such as Open-LLM-VTuber show that desktop-pet style companions are feasible.

Q34. How will memory work, and how is it separated between users?

A: Each user has their own records in the database keyed by user ID, storing preferences and items the user asks the assistant to remember. Queries are always scoped by user ID, and stored content can be viewed and deleted by the user.

Q35. How will reminders work?

A: Reminders are stored with a schedule (one-time, daily or recurring rule). A scheduler in the backend checks due reminders and triggers a native desktop notification. Natural-language requests such as "remind me every day at 8 PM to study" are parsed into that structure.

Q36. What happens if the app is closed when a reminder is due?

A: We plan to keep the app in the system tray so it continues running, and show missed reminders on next launch [decide].

Q37. Why is account recovery needed in a desktop app?

A: Because memory and reminders are per-user, losing access means losing data. Recovery can be implemented with a verification email or security questions [decide]; we will choose based on available infrastructure.

Q38. How will you test the software itself, apart from AI evaluation?

A: Unit tests for validators, scheduling and data access; integration tests for the pipeline using recorded sample screenshots; manual tests for the interface and avatar; and security checks on authentication and data isolation.

Q39. What are the main limitations of your system?

A: Limited application and task coverage; dependence on extraction quality; dependence on an external AI service; a small team-made test set; and screen-based detection that can break when an application's appearance changes. We state these openly and design the evaluation to expose them.

Q40. What would you do with more time?

A: Add more applications, support voice interaction, run a user study measuring perceived usefulness and trust (as in Intent Assistant and Pointer Assistant studies), add local model support for privacy, and extend memory and calendar integration.

F. Tricky and curveball questions

Q41. "Agents now exceed human performance on OSWorld. Isn't your problem solved?"

A: Those results are for agents performing tasks in benchmark environments, often self-reported, and setups differ between submissions, so numbers are not directly comparable. They do not tell us how often an advisory assistant raises false alarms in a user's own application, how much it costs per suggestion, or whether users trust it. Our work targets reliability, cost and trust, and measures them.

Q42. "Why not fine-tune your own model?"

A: Out of scope for time and data reasons. Related work such as OmniParser and ScreenLLM trains specialised models, but our contribution is the validation layer around an existing model, which is a systems and evaluation problem.

Q43. "Your validation just reimplements a linter. Where is the AI value?"

A: The linter finds problems; the AI explains them in context, prioritizes, suggests fixes, answers follow-up questions and handles cases the rules do not cover. Hybrid studies show each part has a different strength: tools give systematic coverage and the model adds contextual reasoning.

Q44. "If validation is so good, why is the AI needed at all?"

A: For explanation, interaction and unstructured tasks. Validation guarantees the evidence is right; the model turns evidence into helpful human language and handles questions.

Q45. "What is your success criterion?"

A: We will define targets after a pilot run, for example a stated precision and F1 level, and a measurable improvement in precision over the raw-screen baseline at acceptable recall and latency [**decide**]. Having the criterion fixed before the final evaluation avoids tuning to the result.

Q46. "What if the supervisor or committee disagrees with the application list?"

A: The list is explicitly marked as to be finalized based on feasibility, validation methods and evaluation needs. We will accept advice and select applications where mistakes can be defined and verified automatically.

Q47. "Who owns the data you send to the AI provider?"

A: We will review the chosen provider's data-use terms before selecting it, send minimal validated content, and avoid personal data in test scenarios. Users are informed that content is sent to an external service.

Q48. "Can you give a concrete example end to end?"

A: The user selects VS Code and starts monitoring. They write a Python function that uses an undefined variable. The system captures the window, detects VS Code and the Python task, extracts the code, and the validator confirms an undefined-name error at a specific line. Only that confirmed finding and the surrounding code go to the AI model, which explains the issue and suggests a fix. The avatar shows a short message with a link to the full chat. If the validator had found nothing, the assistant would have stayed silent.

Part 9: Last-minute cheat sheet

Numbers worth remembering (verify before quoting)

- OSWorld: 369 tasks; human about 72.36%; best model at publication 12.24%; some later agents reported at or above human level (self-reported, setups differ).
- Hybrid static analysis plus LLM (Du et al., ICSE 2026 SEIP): 433 alarms (328 false, 105 true); 94-98% of false positives removed with high recall; 10-20 minutes of manual inspection per false alarm.
- Pointer Assistant (CHI '25): 220 participants, budget planning task; users liked the companion, wanted better assertiveness.
- Intent Assistant (CHI '26): deployment with 22 participants; gentle, non-intrusive nudges.
- AskEase: 12 screen-reader users; improved task success and reduced workload.
- Microsoft Recall: announced May 2024; delayed after privacy backlash; later opt-in and off by default.

Five sentences to have memorised

1. "We validate before we ask the AI, so the AI only sees evidence that passed a check."
2. "Our assistant advises; it never controls the user's computer."
3. "It monitors only the window the user selects, and the user can stop it at any time."
4. "When it is not confident, it stays silent, because false alarms destroy trust."
5. "We prove it with precision, recall and F1 against a raw-screen baseline, not with demos."

Vocabulary you should be able to define

GUI agent; grounding; multimodal LLM; OCR; accessibility tree; hallucination; external validator; abstention; precision; recall; F1; false positive rate; ground truth; baseline; threat to validity; proactive agent.

Things to avoid saying

- "Nobody has done this." Say "in the work we reviewed."
- "It will be 100% accurate." Say what you will measure.
- "We will support all applications." Say the limited set.
- "The AI will just figure it out." Always point to the validation and evaluation.
- Quoting numbers you have not checked in the original paper.

Questions you can ask the committee (shows initiative)

- "Would you prefer we prioritize breadth of applications or depth of evaluation?"
- "Is a comparison against a raw-screenshot baseline sufficient, or should we include a user study?"
- "Do you have a preference between an external AI API and a local model for privacy?"

Presentation tips

- Put the pipeline diagram and the problem-solution table on slides; those two carry most questions.
- Prepare one live or recorded mini-demo, for example a seeded mistake in VS Code, and a fallback screenshot in case of network failure.
- Split speaking between teammates and let each answer questions about the parts they built.
- If you do not know, say: "We have not evaluated that yet. Our plan is X, and we would measure it by Y."

Part 10: References and further reading

Open each source and check the details before citing. Items from my search are listed with the identifiers I found.

Surveys and benchmarks

[1] R. Zhang et al., "GUI Agents: A Survey," Findings of ACL 2025, pp. 22522-22538.

<https://aclanthology.org/2025.findings-acl.1158.pdf>

[2] "A Survey on (M)LLM-Based GUI Agents," arXiv:2504.13865, 2025.

<https://arxiv.org/abs/2504.13865>

[3] "Large Language Model-Brained GUI Agents: A Survey," arXiv:2411.18279.

<https://arxiv.org/abs/2411.18279>

[4] "GUI Agents with Foundation Models: A Comprehensive Survey," arXiv:2411.04890.

<https://arxiv.org/abs/2411.04890>

[5] T. Xie et al., "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," NeurIPS 2024 (arXiv:2404.07972). Project and leaderboard:

<https://github.com/xlang-ai/OSWorld>

Screen understanding

[6] Y. Lu, J. Yang, Y. Shen, A. Awadallah, "OmniParser for Pure Vision Based GUI Agent,"

arXiv:2408.00203, 2024. <https://arxiv.org/abs/2408.00203>

[7] "ScreenLLM: Stateful Screen Schema for Efficient Action Understanding and Prediction," arXiv:2503.20978, 2025. <https://arxiv.org/abs/2503.20978>

Proactive and screen-aware assistants

[8] "State Your Intention to Steer Your Attention: An AI Assistant for Intentional Digital Living" (Intent Assistant), arXiv:2510.14513, CHI 2026. <https://arxiv.org/abs/2510.14513>

- [9] T. Prasongpongchai, P. Pataranutaporn, M. Lertsutthiwong, P. Maes, "Talk to the Hand: an LLM-powered Chatbot with Visual Pointer as Proactive Companion for On-Screen Tasks," CHI 2025. <https://hdl.handle.net/1721.1/164288>
- [10] "ProAgentBench: Evaluating LLM Agents for Proactive Assistance with Real-World Data," arXiv:2602.04482, 2026. <https://arxiv.org/abs/2602.04482>
- [11] N. Chen et al., "From Struggle to Success: Context-Aware Guidance for Screen Reader Users in Computer Use" (AskEase), arXiv:2601.18092, 2026. <https://arxiv.org/abs/2601.18092>
- [12] D. Elenius, H. Bui et al., "A context-aware personal desktop assistant," AAMAS 2008 (demo), pp. 1679-1680.

Hallucination, validation and hybrid error detection

- [13] "LLM-based Agents Suffer from Hallucinations: A Survey of Taxonomy, Methods, and Directions," arXiv:2509.18970. <https://arxiv.org/abs/2509.18970>
- [14] "Large Language Models Hallucination: A Comprehensive Survey," arXiv:2510.06265. <https://arxiv.org/abs/2510.06265>
- [15] X. Du et al., "Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry," ICSE 2026 (SEIP), arXiv:2601.18844. <https://arxiv.org/abs/2601.18844>
- [16] M. Iranmanesh et al., "ZeroFalse: Improving Precision in Static Analysis with LLMs," arXiv:2510.02534. <https://arxiv.org/abs/2510.02534>

Metrics

- [17] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," Information Processing & Management, vol. 45, no. 4, pp. 427-437, 2009.

Products, projects and privacy

- [18] Open-LLM-VTuber (open-source AI companion with desktop-pet mode). <https://github.com/Open-LLM-VTuber/Open-LLM-VTuber>
- [19] Clicky for Windows (open-source screen-aware AI buddy). <https://github.com/Bitshank-2338/clicky-windows>
- [20] Microsoft Copilot Vision (Desktop Share) coverage, e.g. ETCentric: <https://www.etcentric.org/?p=194487>
- [21] Microsoft Recall privacy coverage: BBC (Z. Kleinman), "Microsoft re-launches 'privacy nightmare' AI screenshot tool"; Dark Reading, "Microsoft's 'Recall' feature draws criticism from privacy advocates"; Techzine, "Microsoft Recall is still a privacy nightmare."

Suggested next reading for your own literature review

- Earlier GUI-agent and web-agent papers already in your draft: ReAct, Mind2Web, SeeAct, CogAgent.
- HCI work on interruptions, proactive assistance and trust in AI suggestions.
- Documentation for your chosen tools (Electron capture API, OCR library, linter) so you can justify each design choice.

Caveat on sources. Several items above come from abstracts and summaries (some via aggregator sites), and leaderboard and vendor figures are self-reported. Treat this guide as a map, and confirm every number and claim in the primary source before it appears on a slide.

Part 11: Links for the sources I only mentioned or skimmed

These are the papers that appeared in this guide without a link, or that I only know from abstracts. I confirmed each link below through a search, but I still did not read the full text. Open them before you cite.

Papers from your own draft (listed only as "further reading" above)

Paper	Venue	Links	What it is useful for
S. Yao et al., <i>ReAct: Synergizing Reasoning and Acting in Language Models</i>	ICLR 2023	arXiv: https://arxiv.org/abs/2210.03629 ; project page: https://react-lm.github.io	Reasoning interleaved with actions; reports reduced hallucination and more interpretable, trustworthy behaviour than methods without reasoning traces.

Paper	Venue	Links	What it is useful for
X. Deng et al., <i>Mind2Web: Towards a Generalist Agent for the Web</i>	NeurIPS 2023 (Datasets and Benchmarks)	arXiv: https://arxiv.org/abs/2306.06070 ; NeurIPS: https://papers.neurips.cc/paper_files/paper/2023/hash/5950bf290a1570ea401bf98882128160-Abstract-Datasets_and_Benchmarks.html ; project: https://osu-nlp-group.github.io/Mind2Web	Dataset of 2,000+ tasks across 137 websites. Also shows that filtering large inputs with a small model first improves the LLM, which supports your "filter before the AI" idea.
B. Zheng et al., <i>GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)</i>	ICML 2024	arXiv: https://arxiv.org/abs/2401.01614 ; PMLR: https://proceedings.mlr.press/v235/zheng24e.html ; code: https://github.com/OSU-NLP-Group/SeeAct	Shows that grounding (linking the model's plan to the right screen element) is the main challenge; the best grounding combined HTML structure and visuals. Good support for a hybrid design.
W. Hong et al., <i>CogAgent: A Visual Language Model for GUI Agents</i>	CVPR 2024	arXiv: https://arxiv.org/abs/2312.08914 ; CVF: https://openaccess.thecvf.com/content/CVPR2024/html/Hong_CogAgent_A_Visual_Language_Model_for_GUI_Agents_CVPR_2024_paper.html ; code: https://github.com/THUDM/CogAgent	An 18B vision-language model that reads GUIs from screenshots at high resolution; reported to beat text-only methods that consume extracted HTML on Mind2Web and AITW.

Other sources in the guide that needed links

Source	Links
T. Xie et al., <i>OSWorld</i> (NeurIPS 2024)	NeurIPS page: https://neurips.cc/virtual/2024/poster/97468 ; code and leaderboard: https://github.com/xlang-ai/OSWorld ; arXiv: https://arxiv.org/abs/2404.07972 (ID from my memory; confirm it opens the right paper)
M. Sokolova and G. Lapalme (2009), <i>A systematic analysis of performance measures for classification tasks</i> , Information Processing & Management 45(4), 427-437	DOI: https://doi.org/10.1016/j.ipm.2009.03.002
D. Elenius, H. Bui et al., <i>A context-aware personal desktop assistant</i> (PExA), AAMAS 2008 demo	SRI page: https://www.csl.sri.com/papers/warp-demo-2008/ ; ACM DOI listed on that page: http://doi.acm.org/10.1145/1402744.1402762
Prasongpongchai et al., <i>Talk to the Hand</i> (Pointer Assistant), CHI 2025	MIT repository (open access PDF): https://dspace.mit.edu/handle/1721.1/164288
Zhang et al., <i>GUI Agents: A Survey</i> , Findings of ACL 2025	https://aclanthology.org/2025.findings-acl.1158.pdf
<i>A Survey on (M)LLM-Based GUI Agents</i>	https://arxiv.org/abs/2504.13865
<i>OmniParser for Pure Vision Based GUI Agent</i>	arXiv: https://arxiv.org/abs/2408.00203 ; Microsoft Research: https://www.microsoft.com/en-us/research/publication/omniparser-for-pure-vision-based-gui-agent/
<i>Reducing False Positives in Static Bug Detection with LLMs</i> (Du et al.)	https://arxiv.org/abs/2601.18844 ; ICSE 2026 SEIP listing: https://conf.researchr.org/details/icse-2026/icse-2026-software-engineering-in-practice/72/Reducing-False-Positives-in-Static-Bug-Detection-with-LLMs-An-Empirical-Study-in-Ind

Reading status, so you know what to double-check

- **Seen only as abstracts or summaries:** all of the above, plus the surveys and proactive-assistant papers in Part 10.

- **Numbers that need checking in the full paper:** OmniParser's 0.705 to 0.938 figure came from a third-party review, not the paper itself; the OSWorld scores above the human baseline come from vendor and leaderboard pages; the 94-98% false-positive reduction should be checked against the specific models and datasets in the Du et al. tables.
- **SeeAct detail:** the abstract's live-website success figure appears as 50% in one version and 51.1% in another (v1 vs the published version). Quote the figure from the version you cite.
- **Suggested order for a quick read:** abstract, introduction, the results table and the limitations section of each paper, about 15 minutes each. That is enough to answer most committee questions about it.